

ЛАБОРАТОРНАЯ РАБОТА №2

Проблема собственных значений матрицы

Студенты: Альтшуль Даниил Евгеньевич (ИСУ 505003)

Вохмяков Данила Никитич (ИСУ 504499)

Группа: J3117

Цель работы

Исследовать методы вычисления сингулярного разложения матриц на примере обработки изображений, сравнить эффективность степенного метода и метода вращений Якоби, изучить применение усечённого SVD для сжатия данных.

Теоретические сведения

2.1 Сингулярное разложение (SVD)

Сингулярное разложение матрицы $A \in \mathbb{R}^{m \times n}$ — это представление вида:

$$A = U \Sigma V^T$$

где $U \in \mathbb{R}^{m \times m}$ и $V \in \mathbb{R}^{n \times n}$ — ортогональные матрицы, $\Sigma \in \mathbb{R}^{m \times n}$ — диагональная матрица с неотрицательными сингулярными числами $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ на диагонали.

Усечённое SVD:

$$A_k = U_k \Sigma_k V_k^T, \quad k \ll \min(m, n)$$

позволяет аппроксимировать матрицу A с меньшим числом параметров. При этом ошибка аппроксимации равна следующему сингулярному числу:

$$\|A - A_k\| = \sigma_{k+1}$$

2.2 Степенной метод

Степенной метод позволяет последовательно находить сингулярные числа и соответствующие векторы. На каждой итерации:

1. Выбирается случайный начальный вектор v
2. Итеративно вычисляется: $u = Av$, $\sigma = \|u\|$, $u = u/\sigma$, $v = A^T u$, $v = v/\|v\|$
3. После сходимости вычитается найденный вклад: $A \leftarrow A - \sigma u v^T$

Сложность для нахождения k сингулярных чисел: $\mathcal{O}(k \cdot N \cdot m \cdot n)$, где N — число итераций.

2.3 Метод вращений Якоби

Метод Якоби применяется к симметричной матрице $B = A^T A$ и итеративно обнуляет внедиагональные элементы с помощью матриц вращения Гивенса:

$$B \leftarrow J^T B J$$

где J — матрица вращения в плоскости (p, q) . После сходимости сингулярные числа извлекаются как $\sigma_i = \sqrt{\lambda_i}$, где λ_i — диагональные элементы B .

Сложность: $\mathcal{O}(\text{iter} \cdot n^3)$, где n — размер матрицы B .

2.4 Применение SVD для обработки изображений

Цветное изображение 640×640 пикселей представляется тремя матрицами $R, G, B \in \mathbb{R}^{640 \times 640}$. SVD каждого канала позволяет:

- Сжимать изображение, сохраняя только k первых компонент
- Анализировать распределение информации по сингулярным числам
- Восстанавливать изображение с заданной точностью

Коэффициент сжатия при усечённом SVD с k компонентами:

$$\text{сжатие} = \frac{k(m + n + 1)}{m \cdot n}$$

Ход работы

3.1 Описание исходного изображения



Рис. 1: Исходное изображение

Использовано изображение из сети Интернет: персонаж Angry Birds (птица Red) на золотисто-оранжевом фоне. Исходный размер изображения приведён к 640×640 пикселей, формат .jpg.

Особенности изображения: доминирование тёплых оттенков (красный, оранжевый, золотистый), что напрямую отражается в распределении сингулярных чисел по каналам: $R > G > B$.

3.2 Листинг кода

Импорт библиотек и загрузка изображения

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
import pandas as pd

def do_resize(img, w=640, h=640):
    height, width, _ = img.shape
    if w != width and h != height:
        return cv2.resize(img, (w, h))
    return img

img = cv2.imread("hahaha.jpg")
img = do_resize(img, 640, 640)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
R, G, B = cv2.split(img)
```

Реализация степенного метода

```
def svd(A, singular_values, iter):
    U_cols, sigmas, V_cols = [], [], []
    A_curr = A.copy().astype(float)

    for _ in range(singular_values):
        v = np.random.randn(A_curr.shape[1])
        v = v / np.linalg.norm(v)

        for _ in range(iter):
            u = A_curr @ v
            sigma = np.linalg.norm(u)
            u = u / sigma
            v = A_curr.T @ u
            v = v / np.linalg.norm(v)

        U_cols.append(u)
        sigmas.append(sigma)
        V_cols.append(v)
        A_curr = A_curr - sigma * np.outer(u, v) # deflation /
            rank-1 update

    return np.array(U_cols).T, np.array(sigmas), np.array(V_cols).T
```

Сравнение ошибок при различных числах итераций

```
def compare_iterations(channel, channel_name, k=200, iters=[10, 50,
100]):
    print(f'\nканал {channel_name}')
    for it in iters:
        U, Z, V = svd(channel, k, it)
        restored = U @ np.diag(Z) @ V.T
```

```

        error = np.linalg.norm(channel - restored)
        print(f'iter={it}, error={error:.4f}')

for ch, name in zip([R, G, B], ['R', 'G', 'B']):
    compare_iterations(ch, name)

```

Усечённое SVD и восстановление изображения

```

U_r, S_r, V_r = np.linalg.svd(R, full_matrices=False)
U_g, S_g, V_g = np.linalg.svd(G, full_matrices=False)
U_b, S_b, V_b = np.linalg.svd(B, full_matrices=False)

for k in [5, 10, 20, 50, 100]:
    R_k = U_r[:, :k] @ np.diag(S_r[:k]) @ V_r[:, :k]
    G_k = U_g[:, :k] @ np.diag(S_g[:k]) @ V_g[:, :k]
    B_k = U_b[:, :k] @ np.diag(S_b[:k]) @ V_b[:, :k]
    img_restored = np.clip(
        np.stack([R_k, G_k, B_k], axis=2), 0, 255
    ).astype(np.uint8)
    plt.imshow(img_restored)
    plt.title(f'k={k}')
    plt.show()

```

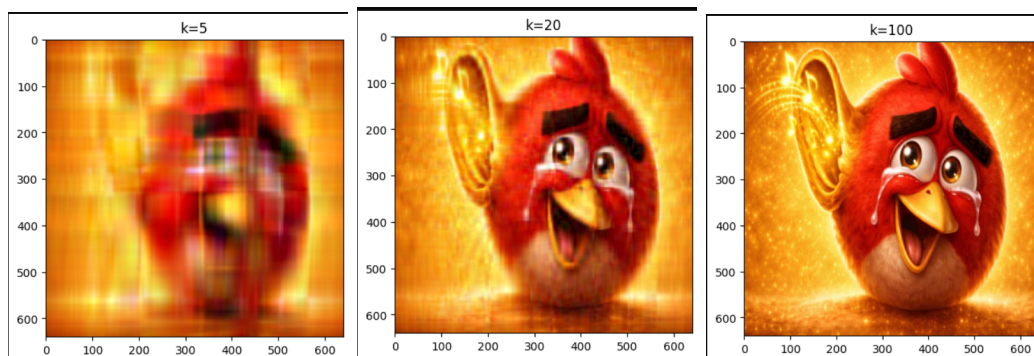


Рис. 2: Восстановленное изображение при $k=5, 20, 100$

Метод вращений Якоби

```

def calc_max_elem_without_diag(B, eps=1e-6): #  $O(n^2)$ 
    max_value, p, q = 0, -1, -1
    n = B.shape[0]
    for i in range(n):
        for j in range(i + 1, n):
            if abs(B[i, j]) > abs(max_value):
                max_value, p, q = B[i, j], i, j
    return (-1, -1) if abs(max_value) < eps else (p, q)

def method_yakobi(A, max_iter=1000, eps=1e-6): #  $O(\text{iter} * n^3)$ 
    B = np.asarray(A, dtype=float).T @ np.asarray(A, dtype=float)
    n = B.shape[0]
    for _ in range(max_iter):
        p, q = calc_max_elem_without_diag(B)

```

```

if p == -1 or q == -1:
    break
if np.abs(B[p][q]) < eps:
    break
r = (B[q][q] - B[p][p]) / (2 * B[p][q])
t = 1/((r) + np.sqrt(1 + r**2)) if r >= 0 \
    else -1/((-r) + np.sqrt(1 + r**2))
c = 1 / np.sqrt(1 + t**2)
s = t * c
J = np.eye(n)
J[p][p], J[q][q], J[p][q], J[q][p] = c, c, s, -s
B = J.T @ B @ J # O(n^3)
singulars = np.sort(np.sqrt(np.diag(B)))[:-1]
return singulars

```

3.3 Таблица результатов: норма ошибки $\|A - U\Sigma V^T\|$

k	Канал	iter=10	iter=50	iter=100
50	R	2987.7977	2983.3460	2983.3351
	G	4536.0334	4530.5144	4530.4962
	B	4670.6678	4667.2234	4667.2227
200	R	641.0911	639.3838	639.3423
	G	762.7475	759.0092	758.9513
	B	772.1471	765.2477	765.2356

* Используется норма Фробениуса. При $k = 50$ и $k = 200$ показана зависимость точности от числа итераций и количества сингулярных чисел.

Анализ результатов

Как число итераций степенного метода влияет на точность разложения?

С ростом числа итераций ошибка убывает, однако прирост точности незначителен: разница между iter= 10 и iter= 100 составляет менее 1%. После 50 итераций результат практически стабилизируется, увеличение числа итераций сверх 50 нецелесообразно.

Гораздо более значимым фактором является число сингулярных чисел k : при $k = 200$ ошибка в 4–5 раз меньше, чем при $k = 50$. Таким образом, точность разложения определяется прежде всего количеством удержанных компонент.

Какое минимальное число сингулярных чисел обеспечивает приемлемое качество изображения?

По графику сингулярных чисел: кривая резко убывает на первых 50 номерах (значения $> 80\,000$), затем практически выходит на плато. Экспериментально подтверждено, что при $k = 50$ изображение визуально неотличимо от оригинала — птица и фон чётко различимы, детали сохранены.

Коэффициент сжатия при $k = 50$:

$$\frac{50 \cdot (640 + 640 + 1)}{640 \cdot 640} \approx 0,156 \quad (\text{сжатие в } \approx 6,4 \text{ раз})$$

Какой метод оказался более эффективным по числу операций?

- **Степенной метод:** $\mathcal{O}(k \cdot N \cdot m \cdot n)$, где $k = 200$, $N = 100$, матрица $640 \times 640 \Rightarrow \approx 8 \cdot 10^9$ операций
- **Метод Якоби:** $\mathcal{O}(\text{iter} \cdot n^3)$, где $\text{iter} = 1000$, $n = 640 \Rightarrow \approx 2,6 \cdot 10^{11}$ операций

Степенной метод на порядки эффективнее при нахождении первых k сингулярных чисел. Метод Якоби находит все сингулярные числа сразу, но платит за это значительно большей вычислительной стоимостью.

Есть ли различия в распределении сингулярных чисел для разных цветных каналов?

Да. Во всех каналах наблюдается одинаковый характер убывания, однако:

- Канал R имеет наибольшие сингулярные числа в начале — изображение богато красными оттенками (птица и фон), информация концентрируется в первых компонентах
- Канал B имеет наименьшие значения — синего на изображении мало, распределение более равномерное
- Канал G занимает промежуточное положение

Это подтверждается и гистограммой: R-канал имеет пик в области высокой яркости (250+), B-канал — в области тёмных значений (0–30).

Какова степень сжатия данных при использовании усечённого SVD?

k	Размер данных (3 канала)	Сжатие
640 (оригинал)	$3 \times 640 \times 640 = 1\,228\,800$	$1 \times$
200	$3 \times 200 \times (640 + 640 + 1) \approx 768\,600$	$\approx 1,6 \times$
50	$3 \times 50 \times (640 + 640 + 1) \approx 192\,150$	$\approx 6,4 \times$
10	$3 \times 10 \times (640 + 640 + 1) \approx 38\,430$	$\approx 32 \times$

Творческое задание (пункт 5)

а) Гистограмма распределения значений пикселей

```
counts_r, bins_r = np.histogram(R.flatten(), bins=256, range=(0, 255))
counts_g, bins_g = np.histogram(G.flatten(), bins=256, range=(0, 255))
counts_b, bins_b = np.histogram(B.flatten(), bins=256, range=(0, 255))

plt.figure(figsize=(10, 5))
```

```
plt.plot(bins_r[:-1], counts_r, label='R', color='red')
plt.plot(bins_g[:-1], counts_g, label='G', color='green')
plt.plot(bins_b[:-1], counts_b, label='B', color='blue')
plt.xlabel('yarkost pikselya (0-255)')
plt.ylabel('kol-vo pikseley')
plt.title('gistogramma raspredeleniya')
plt.legend()
plt.show()
```

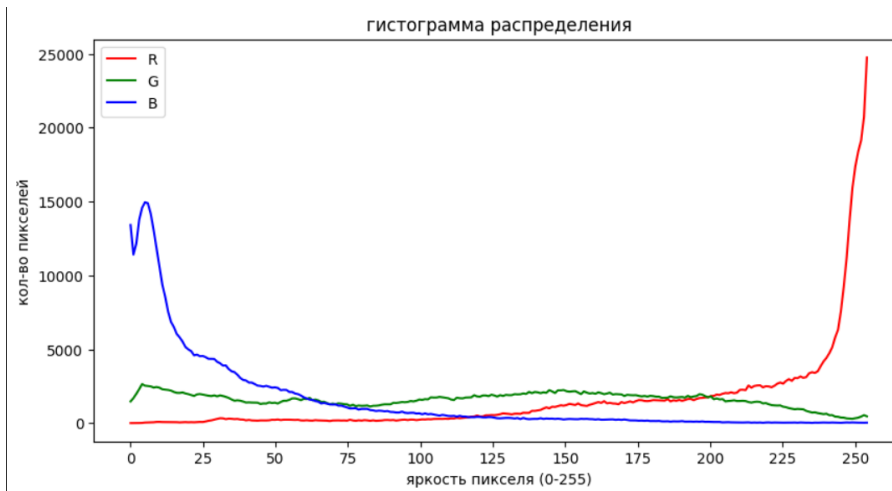


Рис. 3: Гистограмма распределения пикселей по каналам (оригинал)

Анализ: Канал В имеет пик в области яркостей 0–30 (много тёмных пикселей — синего на изображении почти нет). Канал R — пик в области 240–255 (преобладают яркие красные оттенки). Канал G распределён равномерно — зелёный присутствует умеренно в составе золотистого фона (красный + зелёный = жёлтый).

б) Влияние SVD на распределение яркости

При малых k (5, 10) гистограмма сглаживается: пики исчезают, распределение становится более равномерным. При $k \geq 50$ гистограмма практически совпадает с оригиналом. Вывод: усечённое SVD хорошо сохраняет яркостные характеристики изображения при достаточном числе компонент.

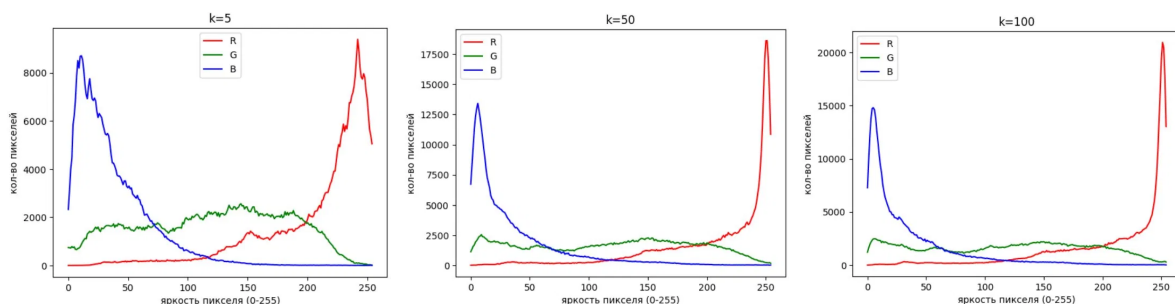


Рис. 4: Гистограммы восстановленного изображения при $k=5, 50, 100$

в) Собственный вариант применения SVD — стилизация изображения

Идея: взять сингулярные числа (Σ) от одного изображения и векторы направлений (U, V) от другого.

Формула обычного SVD: $A = U\Sigma V^T$

В стилизации подменяем компоненты:

$$A_{\text{styled}} = U_A \cdot \Sigma_B \cdot V_A^T$$

где U, V берутся от изображения A (структура, контуры, направления), а Σ — от изображения B (яркость, интенсивность).

U и V хранят информацию о *структуре* изображения — где края, где однородные области. Σ хранит информацию об *интенсивности*. Подменяя Σ , мы переносим «энергетический профиль» одного изображения на структуру другого.

```
# SVD for second image using power method
U_r2, S_r2, V_r2 = svd(R2, singular_values=200, iter=100)

# stylization: U, V from first image, Sigma from second
R_styled = U_r[:, :200] @ np.diag(S_r2) @ V_r[:, :200, :]
G_styled = U_g[:, :200] @ np.diag(S_g2) @ V_g[:, :200, :]
B_styled = U_b[:, :200] @ np.diag(S_b2) @ V_b[:, :200, :]

img_styled = np.clip(
    np.stack([R_styled, G_styled, B_styled], axis=2), 0, 255
).astype(np.uint8)
```

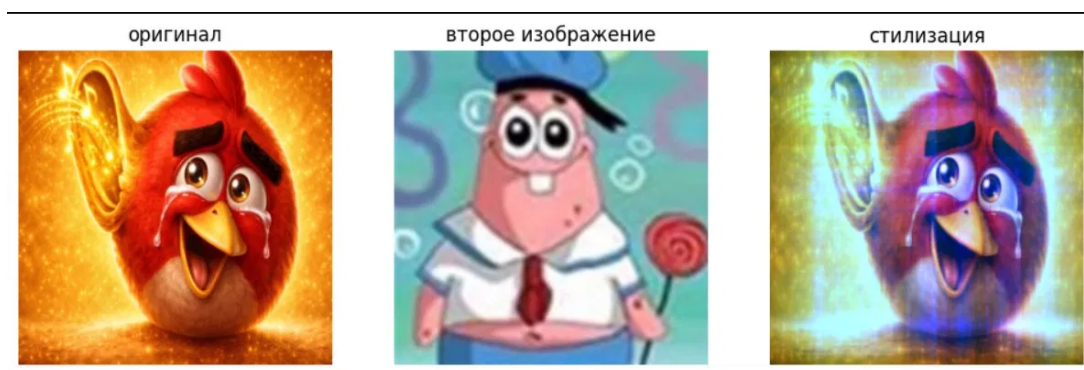


Рис. 5: Стилизация: оригинал (Angry Birds), второе изображение (Патрик), результат стилизации

Выводы о проделанной работе

В ходе лабораторной работы было реализовано и исследовано сингулярное разложение матриц двумя методами. Получены следующие выводы:

- Степенной метод эффективно находит первые k сингулярных чисел: точность стабилизируется уже после 50 итераций, дальнейшее увеличение числа итераций нецелесообразно.

- Качество восстановления определяется числом компонент k , а не числом итераций. При $k = 50$ изображение визуально неотличимо от оригинала при сжатии в $\approx 6,4$ раза.
- Метод Якоби находит все сингулярные числа и даёт результаты, сопоставимые с `numpy.linalg.svd` (точность 10^{-1} – 10^0), однако в ≈ 30 раз тяжелее по числу операций.
- Распределение сингулярных чисел отражает цветовой состав изображения: $R > G > B$, что подтверждается гистограммой яркостей.
- Усечённое SVD является эффективным инструментом сжатия изображений с контролируемой потерей качества.